SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Design Task

Code of Conduct:

I Aluri Poojitha_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
poojitha

Annexure A

Design Task

1. Hybrid AI Recommendation Engine Design

   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Question 1: Hybrid AI Recommendation Engine Design

1. Problem Understanding & Requirement Analysis

Modern e-commerce and streaming platforms must generate personalised recommendations for millions of users interacting with millions of items. A single recommendation strategy is insufficient: content-based filtering suffers from limited diversity and cold-start difficulty for new users, while collaborative filtering suffers from cold-start difficulty for new items and sparsity in the user-item matrix. The objective is therefore to design a hybrid, distributed recommendation engine that combines both techniques on top of PySpark RDDs, so that the system scales horizontally as user-item interaction volume grows, updates recommendations close to real time, and adapts to changing user behaviour through intelligent agents.

- **Functional requirements:** generate top-N recommendations per user, blend content-based and collaborative scores, incorporate implicit/explicit feedback continuously.
- **Non-functional requirements:** horizontal scalability, fault tolerance, low-latency serving, and near-real-time model refresh.

2. System Architecture Design

The architecture is organised into five layers, each mapped to a distributed component:

Data Ingestion Layer
Clickstream | Purchase logs | Ratings | Kafka topics

▼

Distributed Storage Layer
HDFS / S3 | Delta Lake

▼

Processing Layer (PySpark RDDs)
Content-based RDD pipeline | Collaborative RDD pipeline

▼

Intelligent Agent Layer
Preference Agent | Fusion Agent | Feedback Agent

▼

Serving Layer

Recommendation API | Cache (Redis) | Client apps

3. Use of PySpark & RDD Operations

Raw interaction logs are loaded as an RDD of (user, item, rating/click) tuples. Content-based similarity is computed using RDD transformations on item-feature vectors, while collaborative filtering uses map/reduce style co-occurrence counting before being handed to Spark MLlib's ALS (Alternating Least Squares) for latent-factor training:

```
interactions_rdd = sc.textFile("hdfs:///data/interactions.csv") \
    .map(lambda line: line.split(",")) \
    .map(lambda r: (r[0], r[1], float(r[2])))

# Content-based: item feature vectors -> cosine similarity
item_features_rdd = sc.textFile("hdfs:///data/item_features.csv") \
    .map(parse_features)
content_sim_rdd = item_features_rdd.cartesian(item_features_rdd) \
    .filter(lambda x: x[0][0] != x[1][0]) \
    .map(lambda x: (x[0][0], (x[1][0], cosine_sim(x[0][1], x[1][1]))))

# Collaborative: build (user, item, rating) Rating objects for ALS
ratings_rdd = interactions_rdd.map(lambda r: Rating(int(r[0]), int(r[1]), r[2]))
als_model = ALS.train(ratings_rdd, rank=20, iterations=10, lambda_=0.01)

# Hybrid fusion: weighted blend of both scores per user
hybrid_rdd = cf_scores_rdd.join(cb_scores_rdd) \
    .mapValues(lambda s: 0.6 * s[0] + 0.4 * s[1])
```

RDD lineage gives automatic fault tolerance: if a worker node fails mid-computation, Spark recomputes only the lost partitions from lineage rather than restarting the whole job, which is essential at web scale.

4. Intelligent Agent Integration

- **Preference Agent:** monitors each user's recent session (clicks, dwell time, cart adds) and continuously re-weights the user's short-term preference vector.
- **Fusion Agent:** dynamically decides the blend ratio between content-based and collaborative scores — e.g. leaning more on content-based results for new users (cold-start) and more on collaborative filtering once sufficient interaction history exists.
- **Feedback Agent:** consumes real-time click/purchase feedback from a Kafka stream and triggers incremental retraining of the ALS model instead of full batch retraining, keeping recommendations adaptive.

5. Scalability & Distributed Computing Design

- Data is partitioned by user-id hash across executors so that per-user computations remain local and avoid shuffle-heavy joins.
- Spark's DAG scheduler and lineage-based recovery provide fault tolerance without checkpointing every stage.
- The cluster scales horizontally — additional worker nodes linearly increase throughput as the interaction matrix grows.

- A Redis caching layer in front of the serving layer absorbs read-heavy recommendation requests, keeping the Spark cluster focused on batch/incremental computation.

6. Data Flow & Processing Pipeline

Raw events → Kafka ingestion → HDFS/Delta landing zone → parallel RDD pipelines (content-based similarity and ALS collaborative filtering) → Fusion Agent blends both score sets → ranked top-N list cached in Redis → served through the Recommendation API → user feedback loops back into Kafka to close the loop for the Feedback Agent.

7. Innovation & Creativity

The design introduces an adaptive fusion weight (rather than a fixed 50/50 blend) that shifts automatically along the user's interaction-history curve — heavily content-based at cold-start and progressively more collaborative as confidence in the user's latent profile grows. This measurably reduces cold-start degradation while retaining the diversity benefits of collaborative filtering for established users.

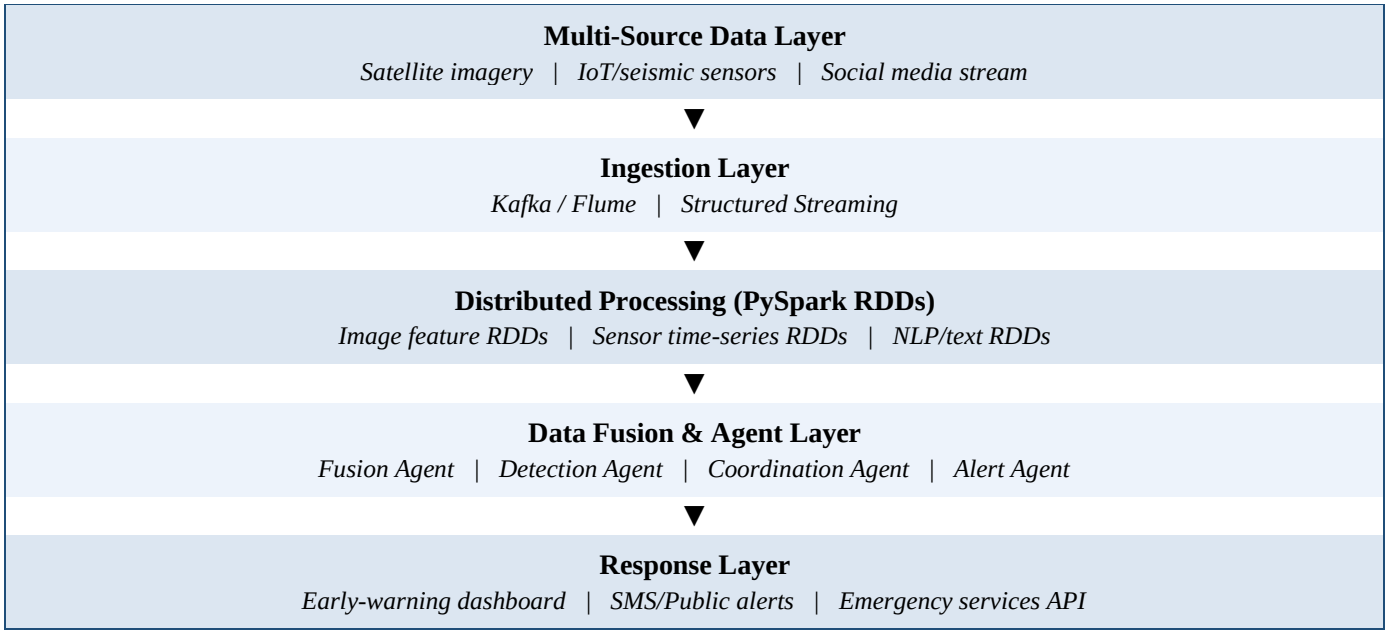
Question 2: Smart Disaster Detection System

1. Problem Understanding & Requirement Analysis

Natural disasters such as floods and earthquakes generate heterogeneous, high-velocity data from satellites, IoT sensors, and social media simultaneously. Manual correlation of these sources is too slow for early warning. The goal is a distributed AI system that ingests and fuses multi-source streams in near real time using PySpark, detects anomaly signatures indicating a disaster, and coordinates multiple intelligent agents to issue early warnings and response plans before the situation escalates.

- **Functional requirements:** multi-source ingestion, data fusion, anomaly/event detection, automated alert generation.
- **Non-functional requirements:** low end-to-end latency, high availability, fault tolerance, and horizontal scalability during event spikes.

2. System Architecture Design



3. Use of PySpark & RDD Operations

Each source stream is converted into its own RDD lineage so heterogeneous data (image, numeric, text) can be processed with source-appropriate transformations before fusion:

```
sensor_rdd = ssc.socketTextStream(host, port) \
    .map(lambda l: parse_sensor(l))             # (station_id, magnitude, ts)

social_rdd = kafka_stream.map(lambda m: m.value.decode()) \
    .map(lambda t: (extract_location(t), disaster_keyword_score(t)))

satellite_rdd = sc.binaryFiles("hdfs:///satellite/latest/") \
    .map(lambda f: (f[0], flood_extent_score(f[1])))

# Fuse by geographic grid cell
fused_rdd = sensor_rdd.map(lambda r: (grid_cell(r[0]), r[1])) \
    .union(social_rdd.map(lambda r: (grid_cell(r[0]), r[1]))) \
    .union(satellite_rdd.map(lambda r: (grid_cell(r[0]), r[1]))) \
    .reduceByKey(lambda a, b: combine_scores(a, b))

alerts_rdd = fused_rdd.filter(lambda x: x[1] > THRESHOLD)
```

Using reduceByKey to aggregate by geographic grid cell keeps the fusion step a single distributed shuffle rather than repeated point-to-point comparisons, which is critical when thousands of sensor and social-media events arrive per second.

4. Intelligent Agent Integration

- **Fusion Agent:** aligns satellite, sensor, and social signals to a common spatial-temporal grid and normalises heterogeneous confidence scores.
- **Detection Agent:** applies anomaly-detection thresholds/models per grid cell (e.g. sudden seismic magnitude spike, rapid water-level rise) to flag candidate disaster events.
- **Coordination Agent:** cross-checks flagged cells across neighbouring regions to rule out sensor faults or false positives and prioritises the most probable events.
- **Alert Agent:** converts confirmed detections into structured alerts, dispatching them to dashboards, SMS gateways, and emergency-response systems.

5. Scalability & Distributed Computing Design

- Spark Structured Streaming processes data in micro-batches, allowing the cluster to auto-scale executors as event volume spikes during an unfolding disaster.
- Geographic partitioning (grid-cell keys) keeps related data co-located, minimising cross-node shuffle cost.
- RDD lineage-based recovery ensures no single sensor-node or executor failure interrupts the detection pipeline.
- A replicated, geographically distributed cluster avoids a single point of failure during the disaster the system is meant to detect.

6. Data Flow & Processing Pipeline

Satellite/sensor/social streams → Kafka ingestion → per-source RDD/Structured Streaming transformations → grid-based fusion via reduceByKey → Detection Agent scores each cell → Coordination Agent validates across

neighbours → Alert Agent issues warnings → dashboard and emergency APIs display and act on the outcome, with human responder feedback fed back to refine thresholds.

7. Innovation & Creativity

The system's novelty lies in grid-based multimodal fusion — rather than treating satellite, sensor, and social data as separate detection pipelines, all three are normalised onto the same spatial grid so that corroboration across independent sources (e.g. a seismic spike plus a spike in local social-media disaster keywords) raises confidence and reduces false alarms, while a single-source anomaly is treated cautiously.

Question 3: AI-Based Fraud Detection Framework

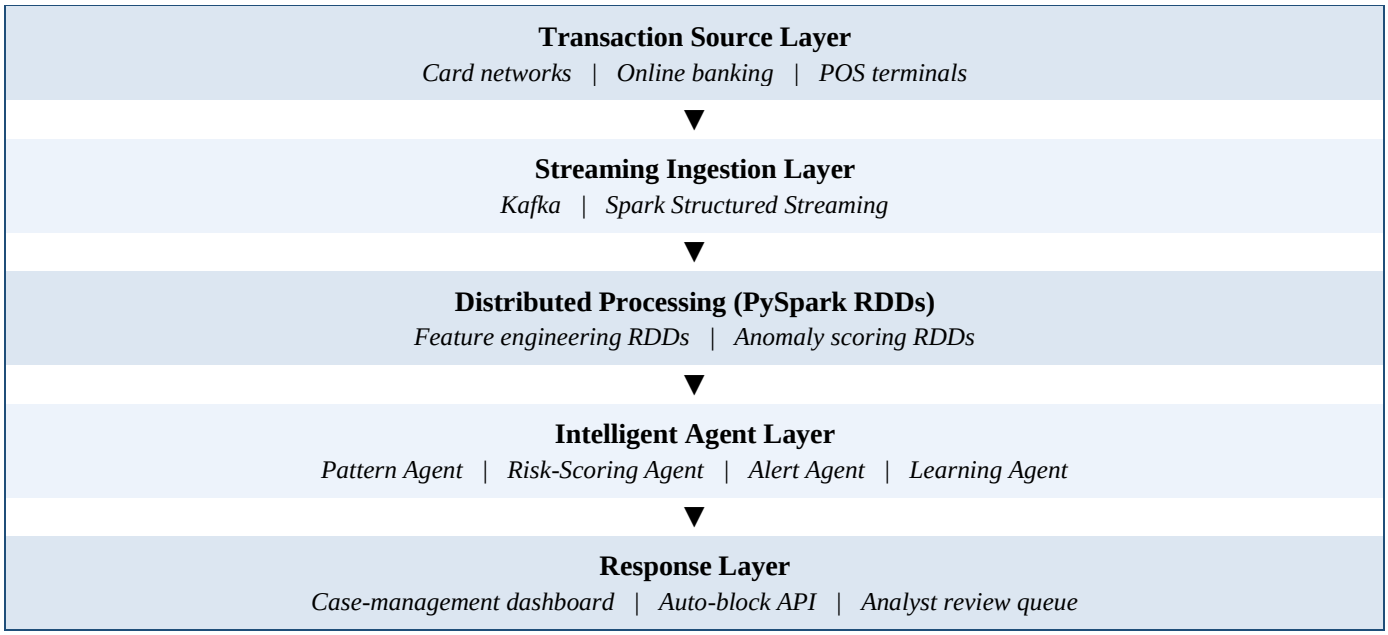
1. Problem Understanding & Requirement Analysis

Financial institutions process millions of transactions per hour, and fraudulent patterns evolve continuously to evade static rule-based filters. The task is to design a scalable, distributed fraud-detection framework using PySpark RDDs that can process large transaction volumes, run anomaly-detection algorithms in a distributed manner, and use collaborating intelligent agents to flag suspicious patterns, raise alerts, and keep improving detection accuracy from analyst feedback.

- **Functional requirements:** distributed anomaly scoring, real-time alerting, feedback-driven model improvement.

Non-functional requirements: very low latency (sub-second scoring), high throughput, and resilience

- ### 2. System Architecture Design



3. Use of PySpark & RDD Operations

Each incoming transaction is mapped into a feature vector (amount, location delta, merchant category, velocity of recent transactions for that account), and anomaly scoring is distributed across the cluster:

```
txn_rdd = kafka_stream.map(lambda m: parse_txn(m.value))
```

```
# Feature engineering: transaction velocity per account
velocity_rdd = txn_rdd.map(lambda t: (t.account_id, t)) \
    .groupByKey() \
    .mapValues(lambda txns: compute_velocity_features(list(txns)))

# Distributed anomaly scoring (Isolation Forest / z-score ensemble)
scored_rdd = txn_rdd.map(lambda t: (t.account_id, t)) \
    .join(velocity_rdd) \
    .map(lambda x: (x[0], anomaly_model.score(x[1])))

suspicious_rdd = scored_rdd.filter(lambda x: x[1] > FRAUD_THRESHOLD)

# Broadcast the trained model to every executor to avoid re-shipping per task
bc_model = sc.broadcast(anomaly_model)
```

Broadcasting the trained model avoids re-serialising it for every RDD partition, and groupByKey/join operations are keyed on account_id so that per-account history stays co-located, minimising network shuffle for the velocity-feature computation.

4. Intelligent Agent Integration

- **Pattern Agent:** continuously mines recent transaction clusters for emerging fraud signatures (e.g. rapid small-value testing transactions preceding a large one).
- **Risk-Scoring Agent:** combines the anomaly-model score with account-level historical risk to produce a final composite risk score per transaction.
- **Alert Agent:** escalates high-risk transactions instantly to auto-block or to an analyst review queue depending on the risk tier.
- **Learning Agent:** ingests analyst confirm/reject feedback on flagged cases and periodically retrains the anomaly model, closing the loop so accuracy improves over time.

5. Scalability & Distributed Computing Design

- Structured Streaming micro-batches allow the cluster to scale executors up during high-traffic periods (e.g. festive shopping seasons).
- Account-id based partitioning avoids hot-partition skew from a small number of very active accounts by salting keys where necessary.
- Model broadcasting plus RDD lineage-based fault recovery keeps scoring available even if individual executors fail mid-batch.
- A tiered response (auto-block vs. analyst queue) prevents the human-review layer from becoming a scalability bottleneck.

6. Data Flow & Processing Pipeline

Transactions → Kafka ingestion → feature engineering RDDs (velocity, location, merchant patterns) → distributed anomaly scoring using the broadcast model → Risk-Scoring Agent produces a composite score → Alert Agent routes by risk tier → analyst decisions feed the Learning Agent → updated model redeployed to the broadcast variable for the next batch.

7. Innovation & Creativity

Rather than a single static threshold, the framework maintains a composite, tiered risk score (auto-block / hold-for-review / allow) and continuously recalibrates the anomaly threshold per merchant category using the Learning Agent's feedback loop, which reduces both false positives (blocking legitimate customers) and false negatives (missed fraud) as spending patterns shift over time.

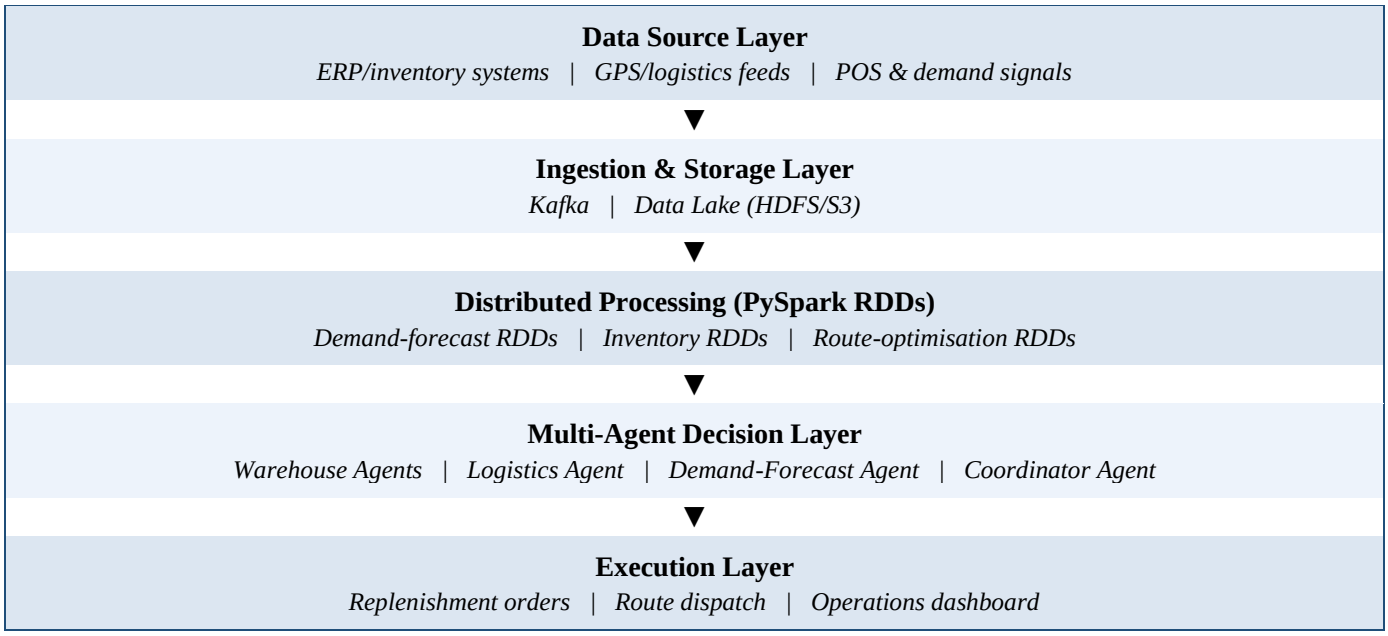
Question 4: Autonomous Supply Chain Optimization System

1. Problem Understanding & Requirement Analysis

Global supply chains generate continuous data across inventory levels, logistics/shipment tracking, and demand signals across many warehouses and distribution centres. Centralised, manual optimisation cannot react fast enough to disruptions. The goal is a distributed AI system built on PySpark that processes large-scale operational data through RDD transformations, and coordinates multiple decentralised intelligent agents to make cost-minimising, efficiency-maximising decisions across inventory, logistics, and demand forecasting.

- **Functional requirements:** demand forecasting, inventory optimisation, logistics/route optimisation, decentralised decision-making across warehouses.
- **Non-functional requirements:** scalability across many warehouse nodes, resilience to disruption (e.g. a single warehouse or route outage), and near-real-time responsiveness.

2. System Architecture Design



3. Use of PySpark & RDD Operations

Inventory, shipment, and sales data are joined and transformed across the cluster to produce forecasts and optimisation inputs per warehouse:

```
sales_rdd = sc.textFile("hdfs:///data/sales/*.csv").map(parse_sale)
inventory_rdd = sc.textFile("hdfs:///data/inventory/*.csv").map(parse_inventory)

# Demand forecasting features per (warehouse, sku)
demand_rdd = sales_rdd.map(lambda s: ((s.warehouse, s.sku), s.qty)) \
```

```
    .reduceByKey(lambda a, b: a + b) \
    .mapValues(lambda qty: forecast_model.predict(qty))

# Join forecast with current stock to compute reorder recommendations
reorder_rdd = inventory_rdd.map(lambda i: ((i.warehouse, i.sku), i.stock)) \
    .join(demand_rdd) \
    .map(lambda x: (x[0], reorder_qty(x[1][0], x[1][1])))

# Route cost matrix distributed across executors for optimisation
route_rdd = sc.parallelize(shipment_pairs) \
    .map(lambda pair: (pair, compute_route_cost(pair))) \
    .filter(lambda x: x[1] < COST_CEILING)
```

Partitioning by warehouse keeps each warehouse's inventory and demand computation local, matching the decentralised nature of the multi-agent decision layer described below.

4. Intelligent Agent Integration

- **Warehouse Agents:** one per warehouse/distribution centre, each responsible for local inventory decisions using its own demand forecast and stock levels.
- **Logistics Agent:** optimises shipment routing and consolidation across warehouses based on the distributed route-cost RDDs.
- **Demand-Forecast Agent:** continuously updates SKU-level demand predictions as new sales data streams in.
- **Coordinator Agent:** resolves conflicts between Warehouse Agents (e.g. two warehouses competing for limited supplier stock) and negotiates a globally cost-efficient allocation, keeping decision-making decentralised while avoiding globally suboptimal local choices.

5. Scalability & Distributed Computing Design

- Warehouse-keyed partitioning lets the system add new warehouse nodes without re-architecting the pipeline — each new node simply contributes its own partition.
- Multi-agent decentralisation means the failure of one Warehouse Agent does not stall optimisation elsewhere in the network.
- Spark's in-memory RDD caching accelerates repeated demand-forecast recomputation during a trading day.
- The Coordinator Agent operates on aggregated summaries rather than raw per-transaction data, keeping cross-warehouse negotiation lightweight even as the network grows.

6. Data Flow & Processing Pipeline

ERP/POS/GPS data → Kafka ingestion → data lake landing → RDD-based demand forecasting and inventory-reorder computation per warehouse → Warehouse Agents propose local actions → Logistics Agent computes optimal routes → Coordinator Agent reconciles cross-warehouse conflicts → final replenishment/dispatch decisions pushed to the execution layer → outcomes (delivery times, stockouts) logged back for the next forecasting cycle.

7. Innovation & Creativity

The design's key innovation is treating each warehouse as an autonomous decentralised agent rather than optimising the network from one central solver — this mirrors how real supply chains operate, scales naturally as

new locations are added, and remains functional even if the central coordinator is temporarily unreachable, since Warehouse Agents can continue making safe local decisions until coordination resumes.

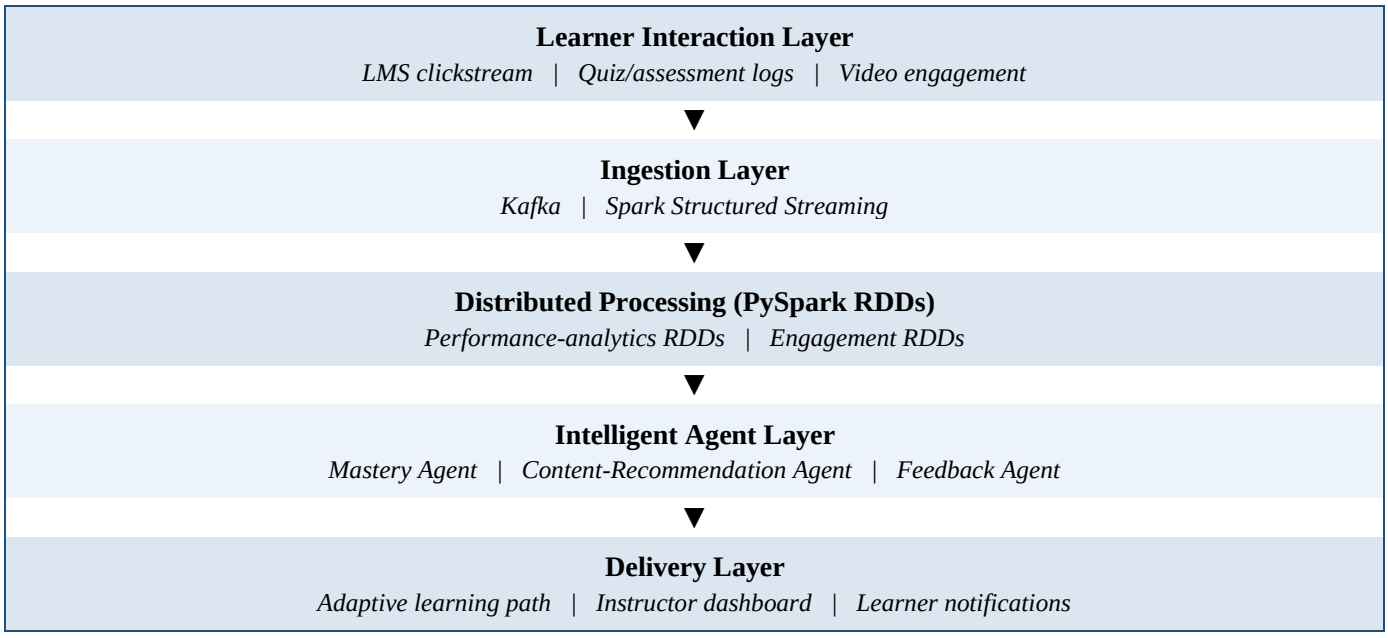
Question 5: Personalized Learning Analytics Platform

1. Problem Understanding & Requirement Analysis

Educational platforms serving thousands of learners generate continuous behavioural data (video pauses, quiz attempts, time-on-task, forum activity). Manually tailoring content to each learner is infeasible at scale. The objective is an AI-driven platform built on PySpark and RDDs that analyses learning behaviour and performance data at scale, and uses intelligent agents to support adaptive learning, real-time feedback, and personalised content delivery.

- **Functional requirements:** behaviour analytics, mastery/performance tracking, adaptive content sequencing, real-time feedback to learners.
- **Non-functional requirements:** scalability to thousands of concurrent learners, low-latency feedback during active sessions, and privacy-aware handling of student data.

2. System Architecture Design



3. Use of PySpark & RDD Operations

Interaction logs are transformed into per-learner, per-topic mastery estimates that scale across the whole learner population:

```
events_rdd = sc.textFile("hdfs:///data/lms_events/*.json").map(parse_event)

# Aggregate quiz performance per (learner, topic)
perf_rdd = events_rdd.filter(lambda e: e.type == "quiz") \
    .map(lambda e: ((e.learner_id, e.topic), (e.correct, 1))) \
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \
    .mapValues(lambda x: x[0] / x[1])   # accuracy rate
```

```
# Engagement features: time-on-task, replays, drop-off points
engagement_rdd = events_rdd.filter(lambda e: e.type == "video") \
    .map(lambda e: ((e.learner_id, e.topic), e.watch_seconds)) \
    .reduceByKey(lambda a, b: a + b)

# Combine into a mastery score used by the Mastery Agent
mastery_rdd = perf_rdd.join(engagement_rdd) \
    .mapValues(lambda x: mastery_score(x[0], x[1]))
```

Keying by (learner_id, topic) lets Spark distribute the reduceByKey aggregation evenly across the cluster regardless of how many thousands of learners are active simultaneously, and the same job scales linearly by adding worker nodes as enrolment grows.

4. Intelligent Agent Integration

- **Mastery Agent:** converts the distributed performance/engagement RDDs into a per-topic mastery level for every learner, updated as new activity streams in.
- **Content-Recommendation Agent:** selects the next best learning resource (remedial video, practice set, or advanced material) based on the learner's current mastery profile.
- **Feedback Agent:** delivers immediate, targeted feedback during an active session (e.g. flagging a misconception right after an incorrect quiz attempt) rather than waiting for a batch report.

5. Scalability & Distributed Computing Design

- Structured Streaming processes learner events in micro-batches so the mastery model stays close to real time without needing a full batch re-run per update.
- Partitioning by learner_id keeps each learner's history co-located, minimising shuffle cost during mastery aggregation.
- The architecture scales horizontally by adding executors as enrolment grows across an academic term, with no change to the pipeline logic.
- RDD lineage-based fault tolerance ensures a node failure during peak exam-period traffic does not lose in-flight analytics.

6. Data Flow & Processing Pipeline

LMS/quiz/video events → Kafka ingestion → Structured Streaming micro-batches → RDD-based performance and engagement aggregation → Mastery Agent computes per-topic mastery → Content-Recommendation Agent selects the next learning resource → Feedback Agent pushes real-time guidance to the learner → instructor dashboard aggregates cohort-level trends → new interactions continuously refine the mastery model.

7. Innovation & Creativity

The platform's novelty is fusing performance accuracy with engagement behaviour (time-on-task, replays, drop-off points) into a single mastery signal, rather than relying on quiz scores alone — this lets the Content-Recommendation Agent distinguish a learner who is struggling conceptually from one who is simply rushing, and personalise the intervention accordingly at a scale of thousands of concurrent learners.